Операционные системы

Лекция 7

Прерывания

План лекции

- Назначение и типы прерываний
- Механизмы обработки прерываний
- Приоритеты прерываний и маскирование прерываний
- Программные прерывания
- Диспетчеризация и приоритеты прерываний в ОС
- Системные вызовы, схемы обработки и режимы выполнения системных вызовов

Прерывания

Прерывания являются основной движущей силой любой операционной системы. Отключите систему прерываний, и «жизнь» в операционной системе немедленно остановится

Как верно было замечено в книге «Ядро Linux в комментариях» – «прерывания названы так весьма удачно, поскольку они прерывают нормальную работу системы»

Система прерываний переводит процессор на выполнение потока команд, отличного от того, который выполнялся до сих пор, с последующим возвратом к исходному коду

Типы прерываний

Механизм прерываний поддерживается аппаратными средствами компьютера и программными средствами операционной системы

В зависимости от источника прерывания делятся на три больших класса:

- Внешние
- Внутренние
- Программные

Внешние прерывания

Внешние прерывания могут возникать в результате:

- Действий пользователя или оператора за терминалом
- Поступления сигналов от аппаратных устройств (сигналов завершения операций ввода-вывода, вырабатываемых контроллерами внешних устройств компьютера, такими как принтер или накопитель на жестких дисках)
- Поступления сигналов от датчиков управляемых компьютером технических объектов

Внешние прерывания

Внешние прерывания называют также **аппаратными**, отражая тот факт, что прерывание возникает вследствие подачи некоторой аппаратурой (например, контроллером принтера) электрического сигнала, который передается (возможно, проходя через другие блоки компьютера, например контроллер прерываний) на специальный вход прерывания процессора

Данный класс прерываний является **асинхронным** по отношению к потоку инструкций прерываемой программы. Аппаратура процессора работает так, что асинхронные прерывания возникают между выполнением двух соседних инструкций, при этом система после обработки прерывания продолжает выполнение процесса, начиная уже со следующей инструкции

Внутренние прерывания

Внутренние прерывания, называемые также **исключениями** (exception), происходят синхронно выполнению программы при появлении аварийной ситуации в ходе обработки некоторой инструкции программы

Примерами исключений являются деление на нуль, ошибки защиты памяти, обращения по несуществующему адресу, попытки выполнить привилегированную инструкцию в пользовательском режиме и т. п. Исключения возникают непосредственно в ходе выполнения тактов команды («внутри» выполнения)

Внутренние прерывания

Исключения делятся на отказы, ловушки и аварийные завершения.

Отказы (**faults**) соответствуют некорректным ситуациям, которые выявляются до выполнения инструкции, например, при обращении по адресу, находящемуся в отсутствующей в оперативной памяти странице (страничный отказ). После обработки исключения-отказа процессор повторяет выполнения команды, которую он не смог выполнить из-за отказа

Ловушки (**traps**) обрабатываются процессором после выполнения инструкции, например, при возникновении переполнения. После обработки процессор выполняет инструкцию, следующую за той, которая вызвала исключение

Аварийные завершения (**aborts**) соответствуют ситуациям, когда невозможно точно определить команду, вызвавшую прерывание. Чаще всего это происходит во время серьезных отказов, связанных со сбоями в работе аппаратуры компьютера

Программные прерывания

Программные прерывания отличаются от предыдущих двух классов тем, что они по своей сути не являются «истинными» прерываниями. Программное прерывание возникает при выполнении особой команды процессора, что имитирует прерывание, то есть переход на новую последовательность инструкций

Причины использования программных прерываний вместо обычных инструкций вызова процедур будут изложены далее, после рассмотрения механизма прерываний

Механизмы обработки прерываний

Прерываниям приписывается приоритет, с помощью которого они ранжируются по степени важности и срочности

О прерываниях, имеющих одинаковое значение приоритета, говорят, что они относятся к одному **уровню приоритета прерываний**

Прерывания обычно обрабатываются модулями операционной системы, так как действия, выполняемые по прерыванию, относятся к управлению разделяемыми ресурсами вычислительной системы: принтером, диском, таймером, процессором и т. п.

IRQ – interrupt request (запрос прерывания) – это уведомление о прерывании, отправляемое из аппаратного устройства центральному процессору

Процедуры, вызываемые по прерываниям, обычно называют **обработчиками прерываний**, или **процедурами обслуживания прерываний** (Interrupt Service Routine, ISR)

Механизмы обработки прерываний

Аппаратные прерывания обрабатываются драйверами соответствующих внешних устройств

Исключения специальными модулями ядра

Программные прерывания – процедурами ОС, обслуживающими системные вызовы

Помимо этих модулей в операционной системе может находиться так называемый ***диспетчер прерываний***, который координирует работу отдельных обработчиков прерываний

Механизмы обработки прерываний

Аппаратная поддержка прерываний осуществляется как на уровне процессора, так и на уровне функциональных блоков компьютера

Функциональный блок компьютера, назначением которого является поддержка прерываний, называется **контроллером прерываний**

Контроллер прерываний является источником сигналов прерываний по отношению к процессору и обработчиком прерываний по отношению к внешним устройствам компьютера

Механизмы обработки прерываний

Существует два основных способа, с помощью которых шины выполняют прерывания: **векторный** (vectored) и **опрашиваемый** (polled)

В обоих способах процессору предоставляется информация об уровне приоритета прерывания на шине подключения внешних устройств

В случае векторных прерываний в процессор передается также информация о начальном адресе соответствующего обработчика прерываний

Механизмы обработки прерываний

Устройствам, которые используют **векторные** прерывания, назначается **вектор прерываний**

Он представляет собой электрический сигнал, выставляемый на соответствующие шины процессора и несущий в себе информацию об определенном, закрепленном за данным устройством номере, который идентифицирует соответствующий обработчик прерываний

Этот вектор может быть *фиксированным*, *конфигурируемым* (например, с использованием переключателей) или *программируемым*

Операционная система может предусматривать процедуру регистрации вектора обработки прерываний для определенного устройства, которая связывает некоторую процедуру обслуживания прерываний с определенным вектором

Механизмы обработки прерываний

При получении сигнала запроса прерывания процессор выполняет специальный цикл подтверждения прерывания, в котором устройство должно идентифицировать себя

В течение этого цикла устройство отвечает, выставляя на шину вектор прерываний. Затем процессор использует этот вектор для нахождения обработчика данного прерывания

Механизмы обработки прерываний

При использовании **опрашиваемых** прерываний процессор получает от запросившего прерывание устройства только информацию об уровне приоритета прерывания

С каждым уровнем прерываний может быть связано несколько устройств и, соответственно, несколько программ – обработчиков прерываний

При возникновении прерывания процессор должен определить, какое устройство из тех, что связаны с данным уровнем прерываний, действительно запросило прерывание

Это достигается вызовом всех обработчиков прерываний для данного уровня приоритета, пока один из обработчиков не подтвердит, что прерывание пришло от обслуживаемого им устройства

Если же с каждым уровнем прерываний связано только одно устройство, то определение нужной программы обработки прерывания происходит немедленно, как и при векторном прерывании

Механизмы обработки прерываний

Механизм прерываний некоторой аппаратной платформы может сочетать векторный и опрашиваемый типы прерываний

Типичным примером является архитектура персональных компьютеров на основе процессоров Intel Pentium

Шины PCI, ISA, EISA или MCA, используемые в этой платформе в качестве шин подключения внешних устройств, поддерживают механизм опрашиваемых прерываний

Контроллеры периферийных устройств выставляют на шину не вектор, а сигнал запроса прерывания определенного уровня IRQ. Однако в процессоре Pentium система прерываний является векторной

Вектор прерываний в процессор Pentium поставляет контроллер прерываний, который отображает поступающий от шины сигнал IRQ на определенный номер вектора

Приоритеты прерываний

Механизм прерываний чаще всего поддерживает **приоритезацию** и **маскирование** прерываний

Приоритезация означает, что все источники прерываний делятся на классы и каждому классу назначается свой уровень приоритета запроса на прерывание

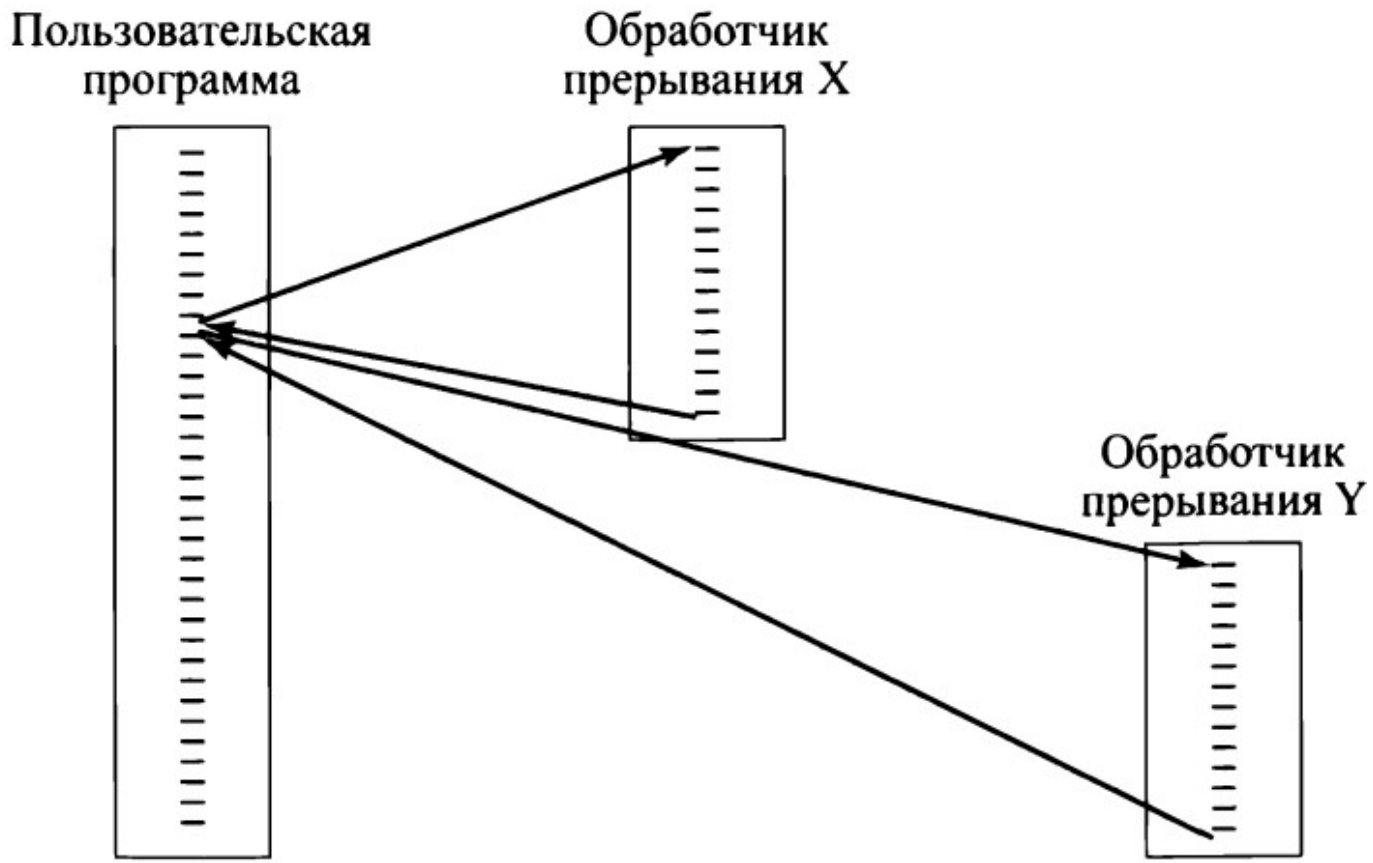
Приоритеты могут обслуживаться как **относительные** и **абсолютные**

Приоритеты прерываний

Обслуживание запросов прерываний по схеме с **относительными** приоритетами заключается в том, что при одновременном поступлении запросов прерываний из разных классов выбирается запрос, имеющий высший приоритет

Однако в дальнейшем при обслуживании этого запроса процедура обработки прерывания уже не откладывается даже в том случае, когда появляются более приоритетные запросы – решение о выборе нового запроса принимается только в момент завершения обслуживания очередного прерывания

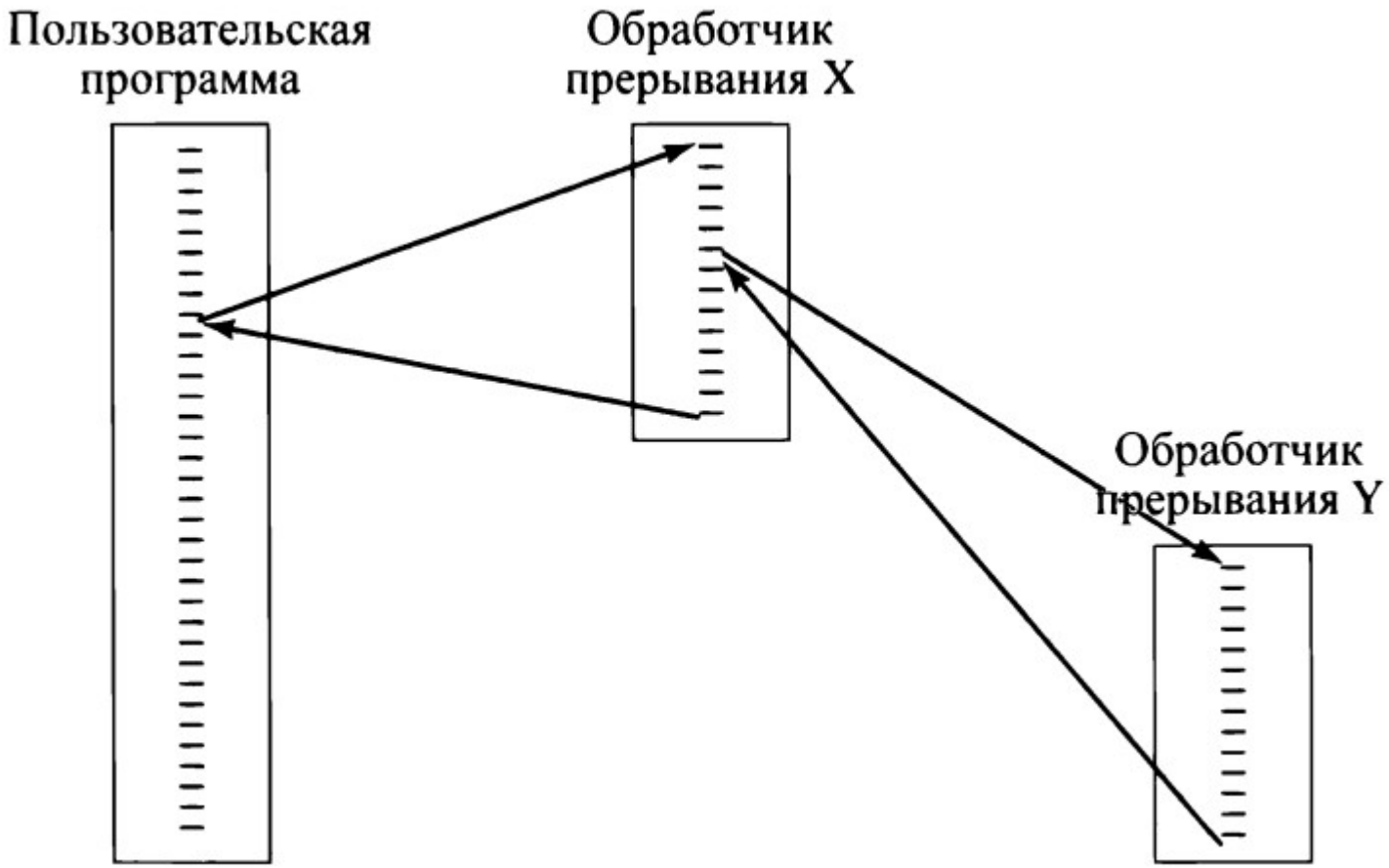
Приоритеты прерываний



Приоритеты прерываний

Если же более приоритетным прерываниям разрешается приостанавливать работу процедур обслуживания менее приоритетных прерываний, это означает, что работает схема приоритезации с **абсолютными** приоритетами

Приоритеты прерываний



Приоритеты прерываний

Если процессор (или компьютер, когда поддержка приоритезации прерываний вынесена во внешний по отношению к процессору блок) работает по схеме **с абсолютными** приоритетами, то он поддерживает **в одном из своих** внутренних **регистров** переменную, фиксирующую уровень приоритета обслуживаемого в данный момент прерывания

При поступлении запроса из определенного класса его приоритет сравнивается с текущим приоритетом процессора, и если приоритет запроса выше, то текущая процедура обслуживания прерываний вытесняется, а при завершении обработки нового прерывания происходит возврат к прерванной процедуре

Маскирование прерывания

Упорядоченное обслуживание запросов прерываний наряду со схемами приоритетной обработки запросов может выполняться **механизмом маскирования** запросов

Собственно говоря, в описанной схеме абсолютных приоритетов тоже выполняется маскирование – при обслуживании некоторого запроса все запросы с равным или более низким приоритетами маскируются, то есть не обслуживаются

Схема маскирования предполагает возможность временного маскирования прерываний любого класса независимо от уровня приоритета

Программные прерывания

Программное прерывание реализует один из способов перехода на подпрограмму с помощью специальной инструкции процессора, такой как INT в процессорах Intel Pentium, trap в процессорах Motorola, syscall в процессорах MIPS или Ticc в процессорах SPARC

При выполнении команды программного прерывания процессор отрабатывает ту же последовательность действий, что и при возникновении внешнего или внутреннего прерывания, но только происходит это в предсказуемой точке программы – там, где программист поместил данную команду

Программные прерывания

Практически все современные процессоры имеют в системе команд инструкции программных прерываний

Одной из причин появления инструкций программных прерываний в системе команд процессоров является то, что их использование часто приводит к более компактному коду программ по сравнению с применением стандартных команд выполнения процедур

Это объясняется тем, что разработчики процессора обычно резервируют для обработки прерываний небольшое число возможных подпрограмм, так что длина операнда в команде программного прерывания, который указывает на нужную подпрограмму, меньше, чем в команде перехода на подпрограмму

Программные прерывания

Другой причиной применения программных прерываний вместо обычных инструкций вызова подпрограмм является возможность смены пользовательского режима на привилегированный одновременно с вызовом процедуры – это свойство программных прерываний поддерживается большинством процессоров

В результате программные прерывания часто используются для выполнения ограниченного количества вызовов функций ядра операционной системы, то есть **системных вызовов**

Планирование прерываний

Прерывания выполняют очень полезную для вычислительной системы функцию – они позволяют реагировать на асинхронные по отношению к вычислительному процессу события

В то же время прерывания создают дополнительные трудности для ОС в организации вычислительного процесса. Эти трудности связаны с непредвиденными переходами управления от одной процедуры к другой, возникающими в результате прерываний от контроллеров внешних устройств

Возможно также возникновение в непредвиденные моменты времени исключений, связанных с ошибками во время выполнения инструкций

Усложняют задачу планирования вычислительных работ и. запросы на выполнение системных функций (системные вызовы) от пользовательских приложений, выполняемые с помощью программных прерываний

Сами модули ОС также часто вызывают друг друга с помощью программных прерываний, еще больше запутывая картину вычислительного процесса

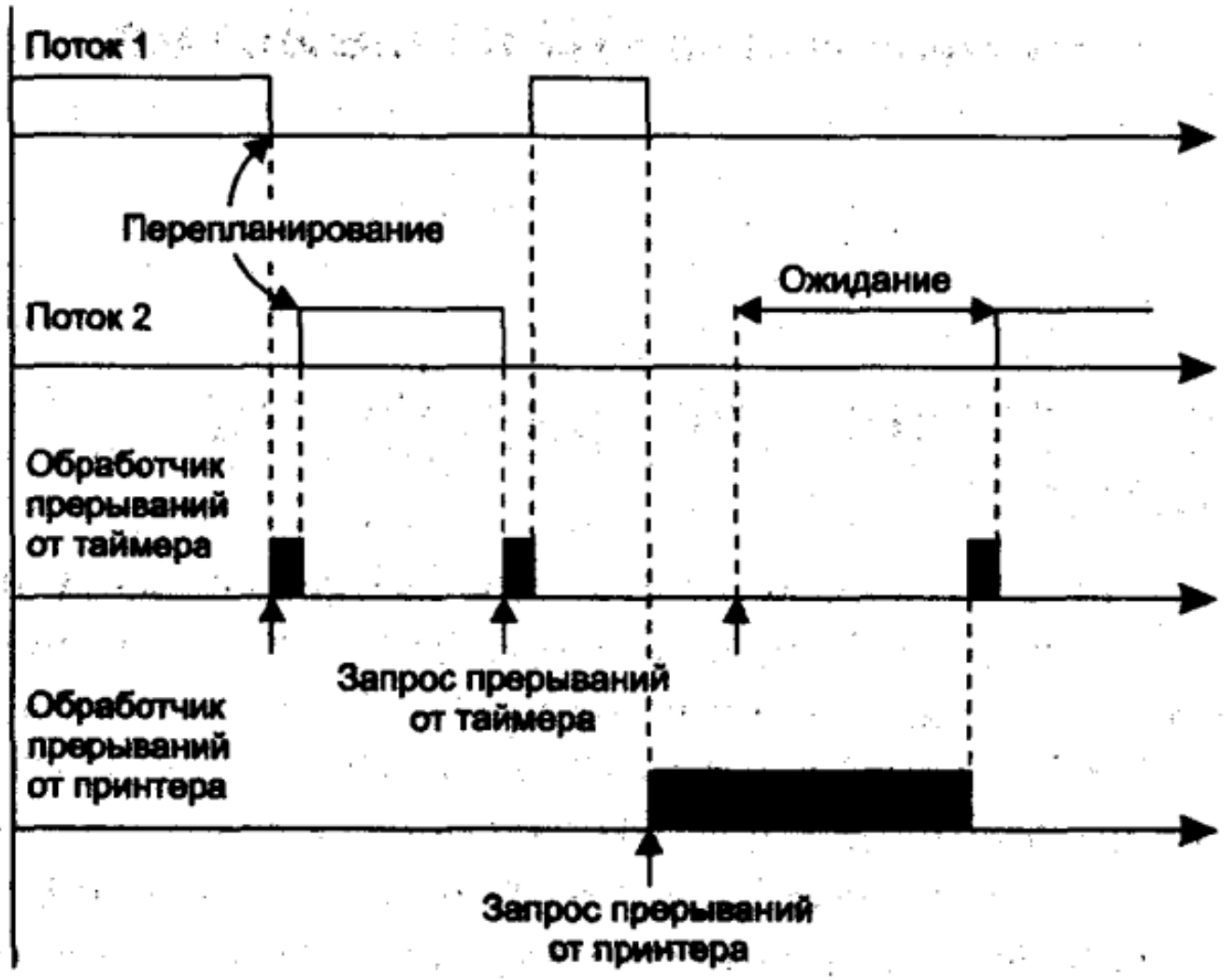
Планирование прерываний

Операционная система не может терять контроль над ходом выполнения системных процедур, вызываемых по прерываниям. Она должна упорядочивать их во времени так же, как планировщик упорядочивает многочисленные пользовательские потоки

Кроме того, сам планировщик потоков является системной процедурой, вызываемой по прерываниям (аппаратным – от таймера или контроллера устройства ввода-вывода, или программным – от приложения или модуля ОС). Поэтому правильное планирование процедур, вызываемых по прерываниям, является необходимым условием правильного планирования пользовательских потоков

Планирование прерываний

В противном случае могут возникать, например, такие ситуации, когда операционная система длительное время занимается не требующей мгновенной реакции



Планирование прерываний

Для упорядочения работы обработчиков прерываний в операционных системах применяется тот же механизм, что и для упорядочения работы пользовательских процессов – **механизм приоритетных очередей**

Все источники прерываний обычно делятся на несколько классов, причем каждому классу присваивается приоритет. В операционной системе выделяется программный модуль, который занимается диспетчеризацией обработчиков прерываний

Этот модуль в разных ОС называется по-разному, но для определенности будем его называть **диспетчером прерываний**

Планирование прерываний

При возникновении прерывания диспетчер прерываний вызывается первым

Он запрещает на небольшое время все прерывания, а затем выясняет причину прерывания. После этого диспетчер сравнивает назначенный данному источнику прерывания приоритет и сравнивает его с текущим приоритетом потока команд, выполняемого процессором

В этот момент времени процессор уже может выполнять инструкции другого обработчика прерываний, также имеющего некоторый приоритет. Если приоритет нового запроса выше текущего, то выполнение текущего обработчика приостанавливается, и он помещается в соответствующую очередь обработчиков прерываний. В противном случае в очередь помещается обработчик нового запроса

Системные вызовы

Системный вызов позволяет приложению обратиться к операционной системе с просьбой выполнить то или иное действие, оформленное как процедура (или набор процедур) кодового сегмента ОС

Для прикладного программиста операционная система выглядит как некая библиотека, предоставляющая некоторый набор полезных функций, с помощью которых можно упростить прикладную программу или выполнить действия, запрещенные в пользовательском режиме, например обменяться данными с устройством ввода-вывода

Организация системных вызовов

Реализация системных вызовов должна удовлетворять следующим требованиям:

- Обеспечивать переключение в привилегированный режим
- Обладать высокой скоростью вызова процедур ОС
- Обеспечивать по возможности единообразное обращение к системным вызовам для всех аппаратных платформ, на которых работает ОС
- Допускать легкое расширение набора системных вызовов
- Обеспечивать контроль со стороны ОС за корректным использованием системных вызовов

Организация системных вызовов

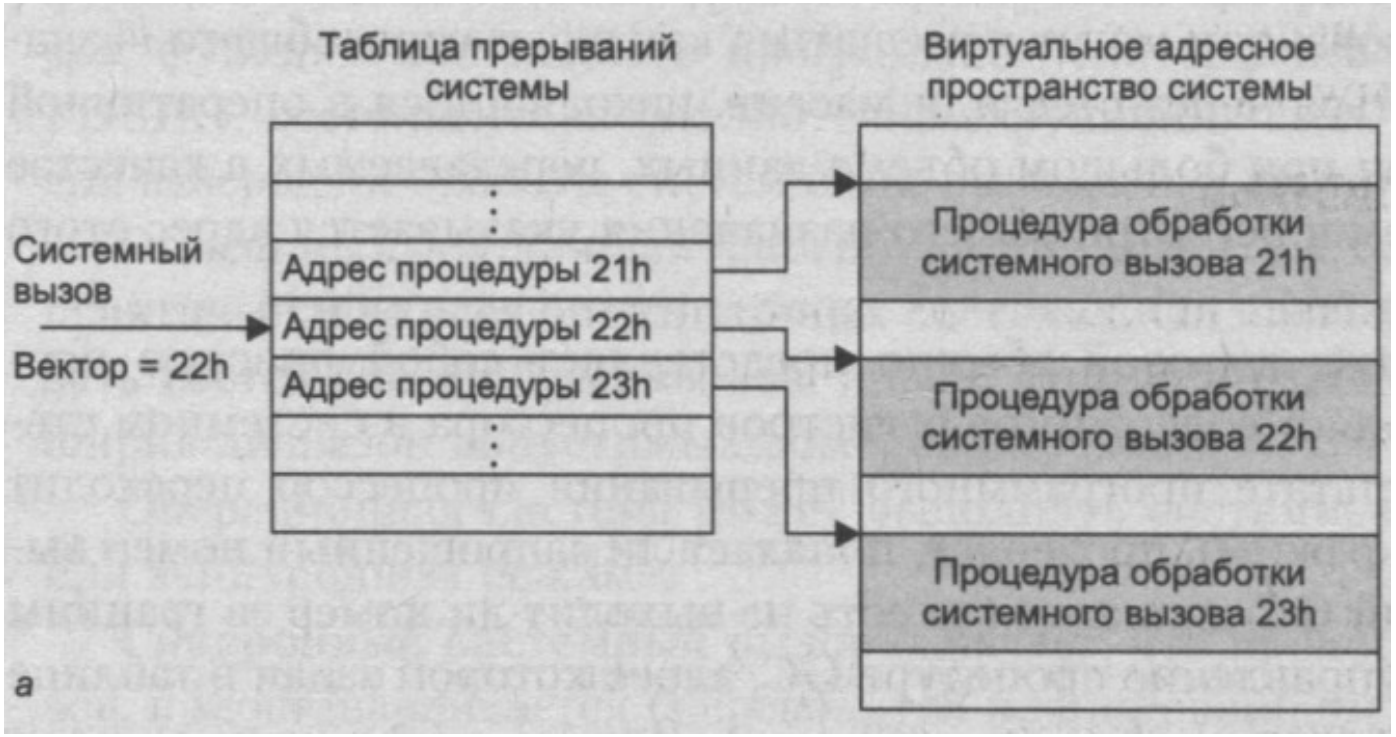
Первое требование для большинства аппаратных платформ может быть выполнено только с помощью механизма программных прерываний. Поэтому будем считать, что остальные требования нужно обеспечить именно для такой реализации системных вызовов

Как это обычно бывает, некоторые из этих требований взаимно противоречивы

Для обеспечения высокой скорости было бы полезно использовать векторные свойства системы программных прерываний, имеющиеся во многих процессорах, то есть закрепить за каждым системным вызовом определенное значение вектора

Организация системных вызовов

Приложение при таком способе вызова непосредственно указывает в аргументе запроса значение вектора, после чего управление немедленно передается требуемой процедуре операционной системы



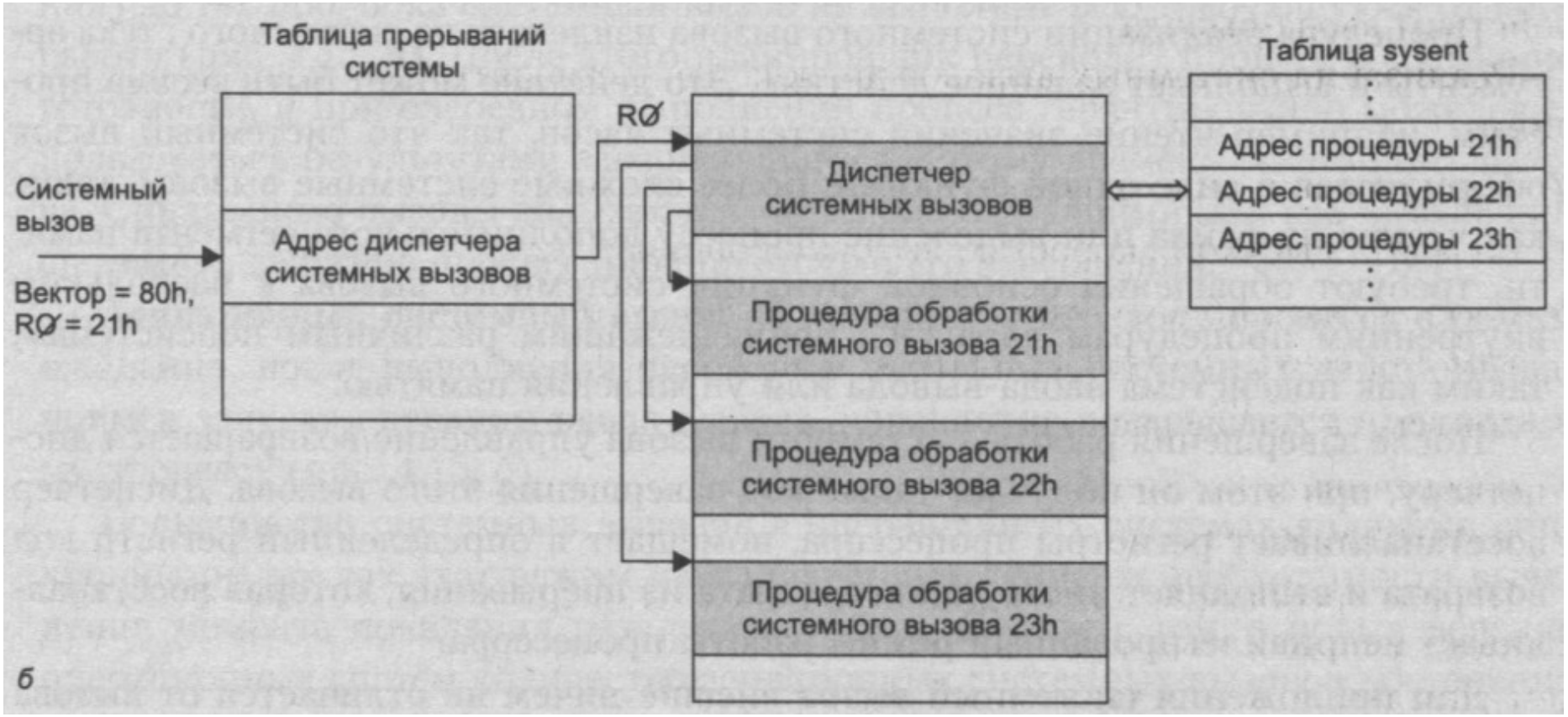
Организация системных вызовов

Однако этот **децентрализованный** способ передачи управления привязан к особенностям аппаратной платформы, а также не позволяет операционной системе легко модифицировать набор системных вызовов и контролировать их использование

Например, в процессоре Pentium количество системных вызовов определяется количеством векторов прерываний, выделенных для этой цели из общего пула в 256 элементов (часть которых используется под аппаратные прерывания и обработку исключений). Добавление нового системного вызова требует от системного программиста тщательного поиска свободного элемента в таблице прерываний, которого к тому же на каком-то этапе развития ОС может и не оказаться

Организация системных вызовов

В большинстве ОС системные вызовы обслуживаются по **централизованной** схеме, основанной на существовании **диспетчера системных вызовов**



Организация системных вызовов

При любом системном вызове приложение выполняет программное прерывание с определенным и единственным номером вектора

Например, ОС Linux использует для системных вызовов команду INT 80h, а ОС семейства Windows NT (при работе на платформе Pentium) – INT 2Eh

Перед выполнением программного прерывания приложение тем или иным способом передает операционной системе номер системного вызова, который является индексом в таблице адресов процедур ОС, реализующих системные вызовы (таблица sysent)

Организация системных вызовов

Способ передачи зависит от реализации, например, номер можно поместить в определенный регистр общего назначения процессора (на рисунке этот регистр условно обозначен R0) или передать через стек (в этом случае после прерывания и перехода в привилегированный режим их нужно будет скопировать в системный стек из пользовательского, это действие в некоторых процессорах автоматизировано)

Диспетчер системных вызовов

Диспетчер системных вызовов обычно представляет собой простую программу, которая:

- Сохраняет содержимое регистров процессора в системном стеке (поскольку в результате программного прерывания процессор переходит в привилегированный режим)
- Проверяет, попадает ли запрошенный номер вызова в поддерживаемый ОС диапазон (то есть не выходит ли номер за границы таблицы)
- Передает управление процедуре ОС, адрес которой задан в таблице адресов системных вызовов

Организация системных вызовов

Процедура реализации системного вызова извлекает из системного стека аргументы и выполняет заданное действие

Это действие может быть весьма простым, например чтение значения системных часов, так что системный вызов оформляется в виде одной функции

Более сложные системные вызовы, такие как чтение из файла или выделение процессу дополнительного сегмента памяти, требуют обращения основной функции системного вызова к нескольким внутренним процедурам ядра ОС, принадлежащим различным подсистемам, таким как подсистема ввода-вывода или управления памятью

Организация системных вызовов

После завершения работы системного вызова управление возвращается диспетчеру, при этом он получает также код завершения этого вызова

Диспетчер восстанавливает регистры процессора, помещает в определенный регистр код возврата и выполняет инструкцию возврата из прерывания, которая восстанавливает непривилегированный режим работы процессора

Организация системных вызовов

Для приложения системный вызов внешне ничем не отличается от вызова обычной библиотечной функции языка C, связанной (динамически или статически) с объектным кодом приложения и выполняющейся в пользовательском режиме

И такая ситуация действительно имеет место – для всех системных вызовов в библиотеках, предоставляемых компилятором C, имеются так называемые «заглушки» (в англоязычном варианте используется термин «stub»)

Каждая «заглушка» оформлена как C-функция, при этом она содержит несколько ассемблерных строк, нужных для выполнения инструкции программного прерывания. Таким образом, пользовательская программа вызывает «заглушку», а та, в свою очередь, вызывает процедуру ОС

Организация системных вызовов

Для ускорения выполнения некоторых достаточно простых системных вызовов, которым, к тому же, не нужно действовать в привилегированном режиме, требуемая работа полностью выполняется библиотечной функцией. В данном случае такую функцию несправедливо называть «заглушкой»

К тому же, такая функция не является системным вызовом, а представляет собой «чистую» библиотечную функцию, выполняющую всю свою работу в пользовательском режиме в виртуальном адресном пространстве процесса, но прикладной программист может об этом и не знать – для него системные вызовы и библиотечные функции выглядят единообразно

Режимы системных вызовов

Описанный табличный способ организации системных вызовов принят практически во всех операционных системах. Он позволяет легко модифицировать состав системных вызовов, просто добавив в таблицу новый адрес и расширив диапазон допустимых номеров вызовов

Операционная система может выполнять системные вызовы в **синхронном** или **асинхронном** режиме

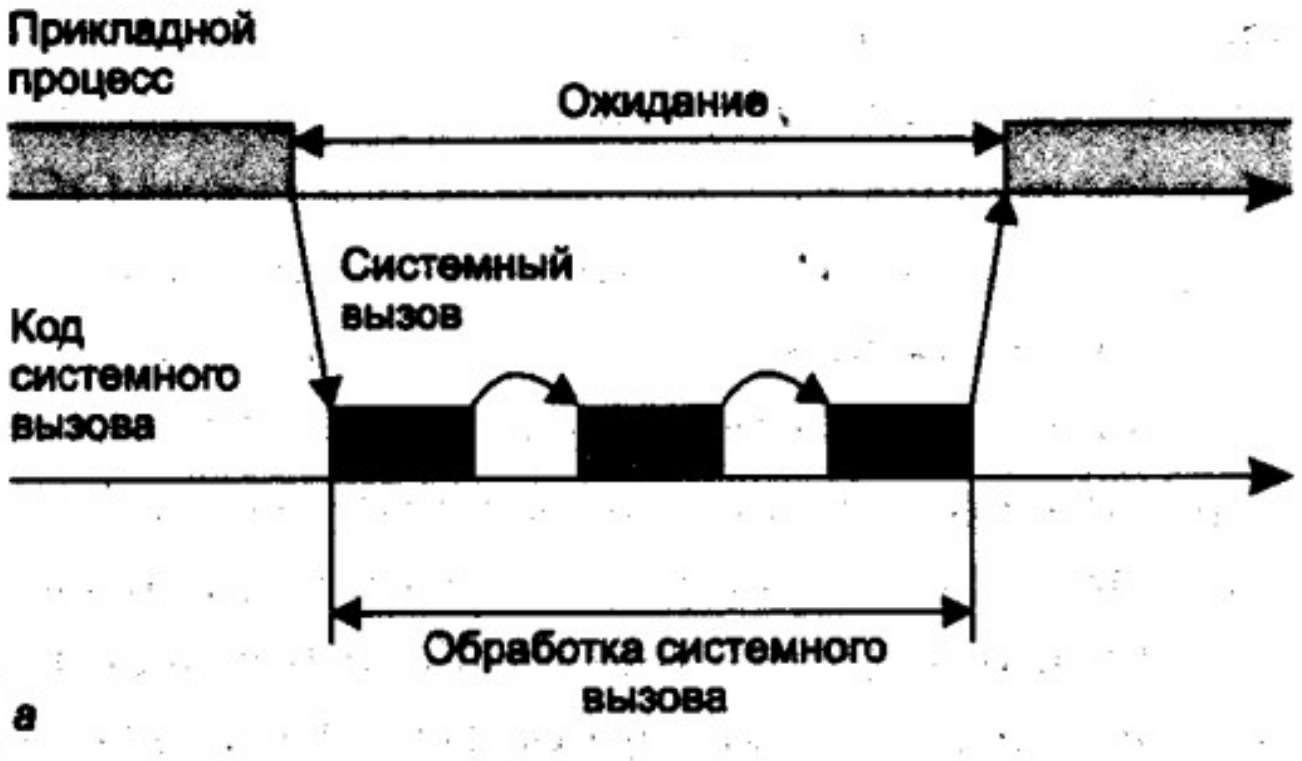
Синхронные системные вызовы

Синхронный системный вызов означает, что процесс, сделавший такой вызов, приостанавливается (переводится планировщиком ОС в состояние ожидания) до тех пор, пока системный вызов не выполнит всю требующуюся от него работу

После этого планировщик переводит процесс в состояние готовности, и при очередном выполнении процесс гарантированно может воспользоваться результатами завершившегося к этому времени системного вызова

Синхронные вызовы называются также блокирующими, так как вызвавший системное действие процесс блокируется до его завершения

Синхронные системные вызовы

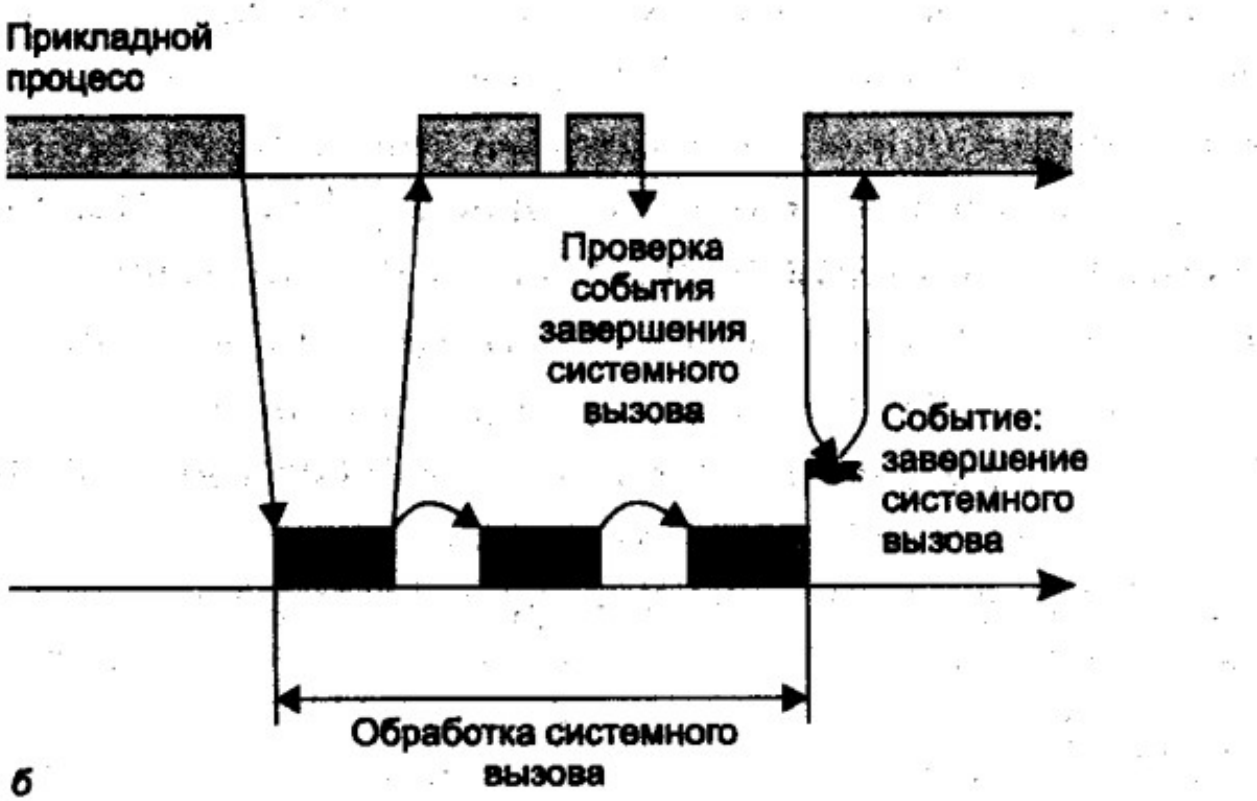


Асинхронные системные вызовы

Асинхронный системный вызов не приводит к переводу процесса в режим ожидания, после выполнения некоторых начальных системных действий, например запуска операции ввода-вывода, управление возвращается прикладному процессу

Большинство системных вызовов в операционных системах являются синхронными, так как этот режим избавляет приложение от необходимости выяснения момента появления результата вызова. Вместе с тем, в новых версиях операционных систем количество асинхронных системных вызовов постепенно увеличивается, что дает больше свободы разработчикам сложных приложений

Асинхронные системные вызовы



Операционные системы

Лекция 7

Прерывания

Прерывания Windows NT

Дальше, необходимо рассмотреть две очень важные концепции ядра Windows, которые играют важную роль в системе ввода/вывода: **уровни запросов прерываний**, или **IRQL** (Interrupt Request Level), и **отложенные вызовы процедур**, или **DPC** (Deferred Procedures Calls)

Термин «IRQL» имеет два разных значения, которые сходятся в некоторых ситуациях:

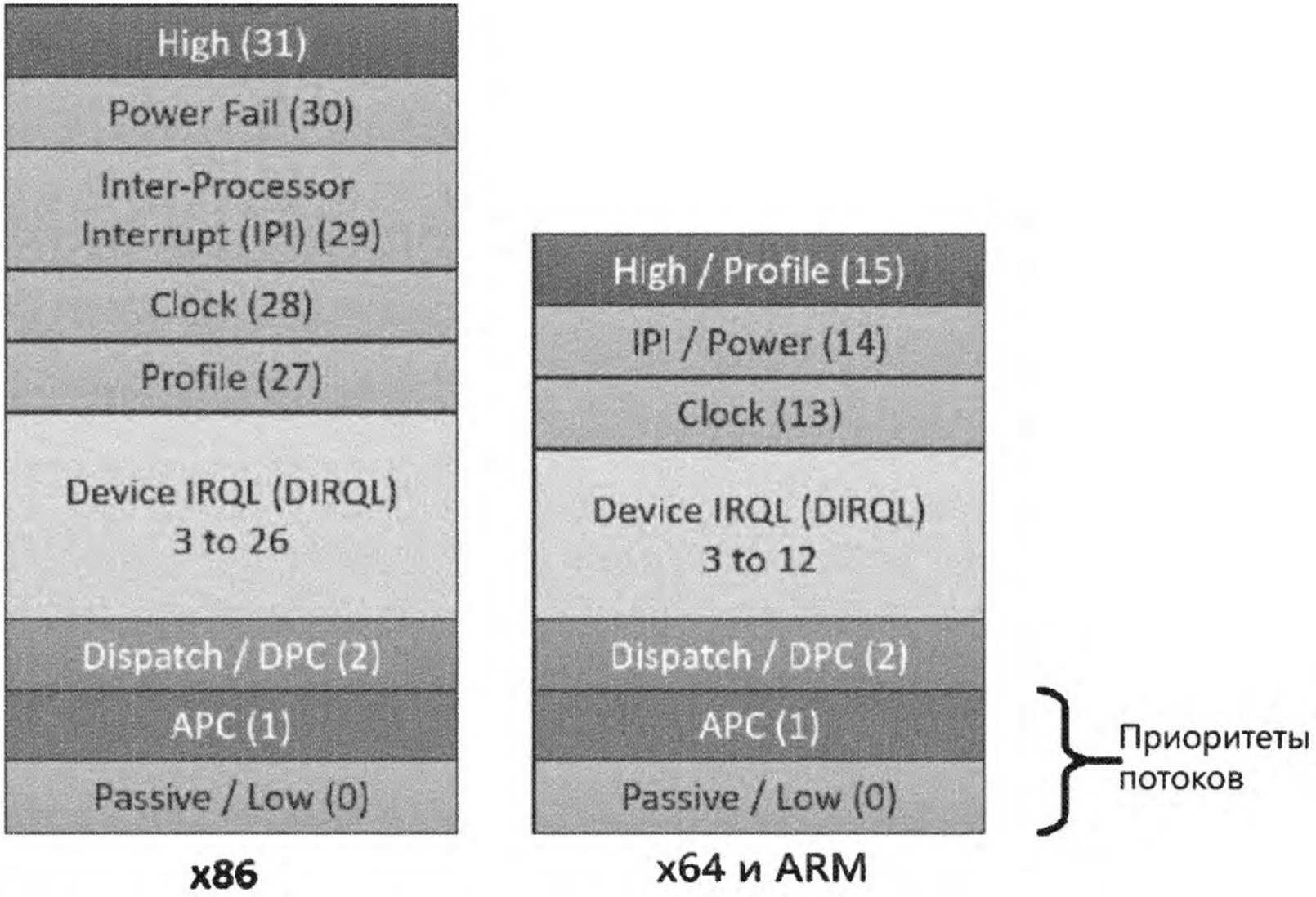
- IRQL – приоритет, назначаемый источнику прерываний от физического устройства. Число задается HAL (при содействии контроллера прерываний, к которому подключаются устройства, требующие обслуживания прерываний)
- У каждого центрального процессора имеется собственный уровень IRQL

Прерывания Windows NT

Фундаментальное правило IRQL гласит, что код с более низким IRQL не может вмешиваться в работу кода с более высоким IRQL, и наоборот – код с более высоким IRQL не может вытеснять код, работающий с более низким IRQL

Обычно уровень IRQL процессора равен 0. Это означает, что в системе не происходит «ничего особенного», а планировщик ядра, который планирует потоки на основании приоритетов. В пользовательском режиме значение IRQL может быть равно только 0. Повысить IRQL из пользовательского режима невозможно. (Поэтому в документации пользовательского режима концепция IRQL вообще не упоминается.)

Прерывания Windows NT



Прерывания Windows NT

Самые важные уровни IRQL в контексте ввода/вывода:

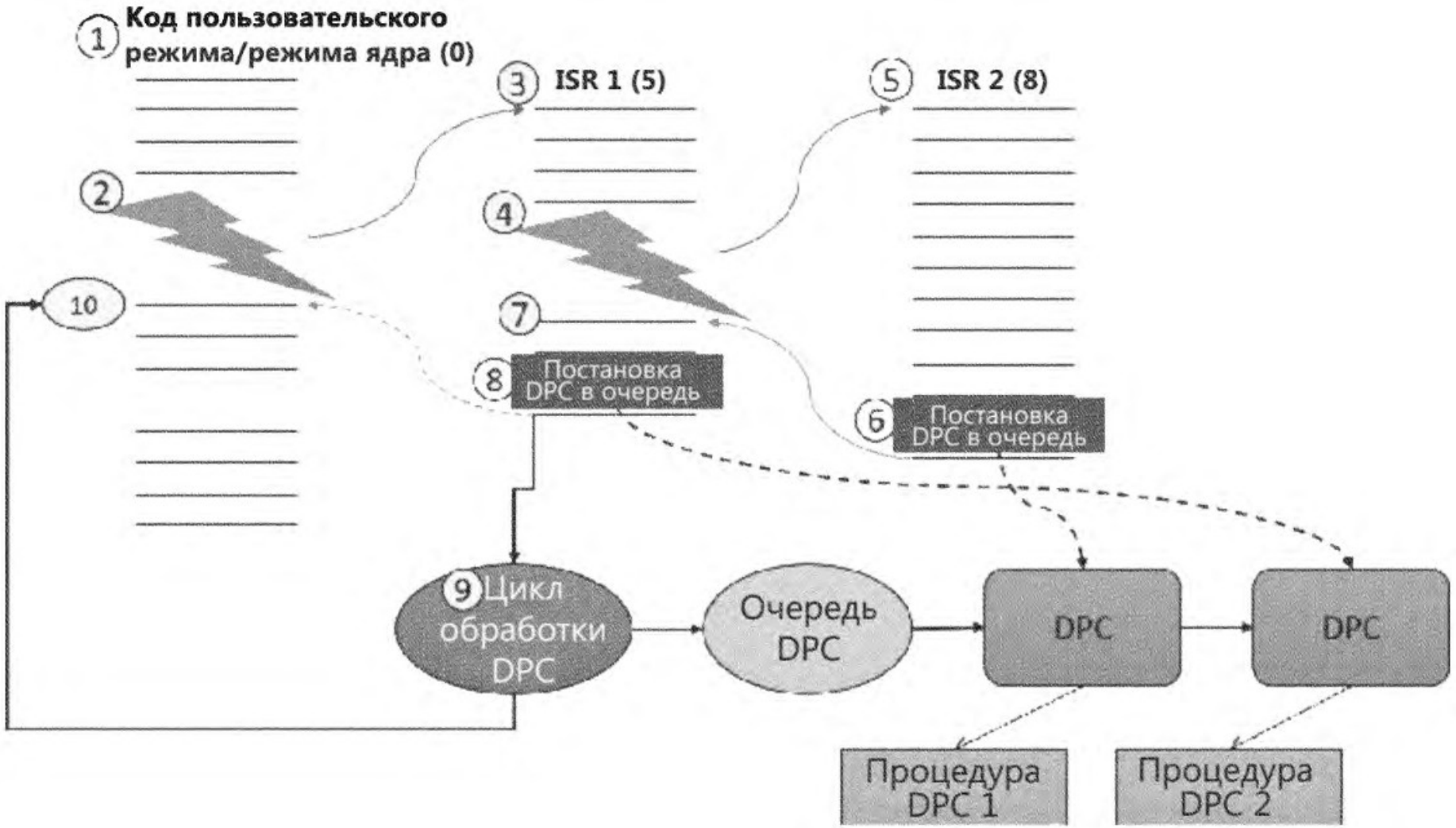
- **Passive** (**0**). Определяется макросом PASSIVE_LEVEL в заголовочном файле WDK wdm.h. Это нормальный уровень IRQL, при котором планировщик ядра работает нормально
- **Dispatch/DPC** (**2**) (DISPATCH_LEVEL). Уровень IRQL, на котором работает планировщик ядра. Это означает, что если поток поднимает текущий уровень IRQL до 2 (или выше), поток фактически получает бесконечный квант и не может быть вытеснен другим потоком. По сути, планировщик не может активизироваться на текущем процессоре, пока уровень IRQL не упадет ниже 2
- **Device IRQL** (3-26 на x86; 3-12 на x64 и ARM) (DIRQL). Эти уровни закрепляются за аппаратными преобразованиями. При поступлении прерывания диспетчер вызывает соответствующую функцию обслуживания прерывания (ISR, Interrupt Service Routine) и повышает ее уровень IRQL до уровня соответствующего прерывания

Прерывания Windows NT

Отложенный вызов процедуры, или **DPC** (Deferred Procedure Call), – объект, инкапсулирующий вызов функции на уровне IRQL DPC_LEVEL (2). Объекты DPC существуют прежде всего для выполнения действий после прерывания, так как выполнение на уровне DIRQL маскирует (а следовательно, задерживает) другие прерывания, ожидающие обработки

Термин «отложенный» в названии означает, что DPC не выполняется немедленно – да и не может, потому что текущий уровень IRQL выше 2. Но когда ISR вернет управление, при отсутствии ожидающих обработки прерываний уровень IRQL процессора падает до 2, и он выполняет накопившиеся вызовы

Прерывания Windows NT



Прерывания Windows NT

Краткая сводка последовательности событий:

1.Некий код пользовательского режима или режима ядра выполняется тогда, когда процессор находится на уровне 0 (на этом уровне проходит большая часть времени)

2.Поступает аппаратное прерывание на уровне IRQL 5. Так как 5 больше 0 (текущий уровень IRQL) состояние процессора сохраняется, IRQL повышает до 5, и вызывается обработчик ISR, связанный с прерыванием

Обратите внимание: переключение контекста при этом не происходит; работает тот же поток, который теперь выполняет код ISR. (Если поток находился в пользовательском режиме, он переключается в режим ядра при поступлении прерывания

Прерывания Windows NT

Краткая сводка последовательности событий:

3.ISR 1 начинает выполняться, когда процессор работает на уровне IRQL 5. На этот момент любые прерывания с IRQL 5 и ниже вмешиваться в обработку не могут

3.Предположим, поступает новое прерывание с IRQL 8. Предположим, система решает, что оно должно быть обработано тем же процессором. Так как 8>5, выполнение снова прерывается, состояние процессора сохраняется, IRQL повышается до 8, и процессор переходит к ISR 2. Заметьте также, что выполнение происходит в том же потоке. Никакое переключение контекста при этом невозможно, потому что планировщик потоков не может активизироваться при IRQL уровня 2 и выше

Прерывания Windows NT

Краткая сводка последовательности событий:

5.Выполняется обработчик ISR 2. До его завершения ISR 2 хотелось бы выполнить дополнительную обработку на более низком уровне IRQL, чтобы прерывания с IRQL менее 8 тоже могли быть обработаны

5.Напоследок ISR 2 вставляет правильно инициализированный объект DPC со ссылкой на функцию драйвера, которая выполняет всю последующую обработку после закрытия прерывания, вызовом функции KeInsertQueueDpc. Затем ISR возвращает управление, и восстанавливается состояние процессора, сохраненное перед входом в ISR 2

Прерывания Windows NT

Краткая сводка последовательности событий:

7.На этой стадии IRQL падает до предыдущего уровня (5), и процессор продолжает выполнение обработчика ISR 1, прерванного ранее

7.Непосредственно перед завершением ISR 1 ставит в очередь собственный объект DPC для выполнения своей последующей обработки. Объекты DPC ставятся в очередь DPC. ISR 1 возвращает управление, восстанавливая состояние процессора, сохраненное перед началом выполнения ISR 1

Прерывания Windows NT

Краткая сводка последовательности событий:

9.В этот момент уровень IRQL должен был бы упасть до старого нулевого значения, чтобы начать обработку прерываний. Но ядро замечает наличие необработанных DPC, поэтому IRQL уменьшается до уровня 2 (DPC_LEVEL), и запускается цикл обработки DPC, который перебирает накопленные DPC и последовательно вызывает каждую процедуру DPC. Когда очередь DPC опустеет, обработка DPC завершается

Прерывания Windows NT

Краткая сводка последовательности событий:

10.Наконец, IRQL уменьшается до 0, состояние процессора снова восстанавливается, и возобновляется выполнение изначально прерванного исходного кода пользовательского режима или режима ядра. И снова следует напомнить, что вся описанная обработка происходит в одном потоке. Этот факт подразумевает, что ISR и процедуры DPC не должны зависеть от конкретного потока (а следовательно, части конкретного процесса) для выполнения своего кода. Поток может быть любым